

```
from __future__ import annotations

import json
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

PROJECT_ROOT = Path(__file__).resolve().parent
SCORED_DATA_PATH = PROJECT_ROOT / "data" / "scored_access_logs.csv"
BASELINE_PATH = PROJECT_ROOT / "models" / "entity_baseline_profiles.csv"
METRICS_PATH = PROJECT_ROOT / "reports" / "detection_metrics.json"

RISK_COLORS = {
    "Low": "#2E8B57",
    "Medium": "#F0A202",
    "High": "#E76F51",
    "Critical": "#B22222",
}

st.set_page_config(
    page_title="SentinelAI | Behavioural Anomaly Detection",
    page_icon="puᳵñēU",
    layout="wide",
)

@st.cache_data
def load_data() -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    events = pd.read_csv(SCORED_DATA_PATH, parse_dates=["timestamp"])
    profiles = pd.read_csv(BASELINE_PATH)

    with open(METRICS_PATH, "r", encoding="utf-8") as file:
        metrics = json.load(file)

    return events, profiles, metrics

def percent(value: float) -> str:
    return f"{value * 100:.2f}%"

def main() -> None:
    st.title("puᳵñēU SentinelAI Security Operations Dashboard")
    st.caption(
        "Explainable behavioural anomaly detection for users, service accounts, and edge devices."
    )

    if not SCORED_DATA_PATH.exists():
        st.error(
            "Scored data was not found. Run `python src\\detector.py` first."
        )
        st.stop()

    events, profiles, metrics = load_data()

    # Sidebar filters
    st.sidebar.header("Alert filters")

    minimum_risk = st.sidebar.slider(
        "Minimum risk score",
        min_value=0,
        max_value=100,
        value=50,
    )

    selected_entity_types = st.sidebar.multiselect(
        "Entity types",
        options=sorted(events["entity_type"].unique()),
        default=sorted(events["entity_type"].unique()),
    )

    available_attack_types = sorted(
        attack
        for attack in events["predicted_attack_type"].unique()
        if attack != "normal"
    )
    selected_attack_types = st.sidebar.multiselect(
        "Predicted attack types",
        options=available_attack_types,
        default=available_attack_types,
    )

    min_date = events["timestamp"].min().date()
    max_date = events["timestamp"].max().date()

    date_range = st.sidebar.date_input(
        "Event date range",
        value=(min_date, max_date),
        min_value=min_date,
        max_value=max_date,
    )

    if isinstance(date_range, tuple) and len(date_range) == 2:
        start_date, end_date = date_range
    else:
        start_date = end_date = date_range

    filtered_events = events[
        (events["entity_type"].isin(selected_entity_types))
        & (events["timestamp"].dt.date >= start_date)
        & (events["timestamp"].dt.date <= end_date)
    ].copy()

    alerts = filtered_events[
        (filtered_events["risk_score"] >= minimum_risk)
        & (filtered_events["predicted_anomaly"] == 1)
    ].copy()

    if selected_attack_types:
        alerts = alerts[
            alerts["predicted_attack_type"].isin(selected_attack_types)
        ]

    alerts = alerts.sort_values("risk_score", ascending=False)

    # Main performance metrics
    total_events = len(filtered_events)
    high_risk_alerts = len(alerts[alerts["risk_score"] >= 75])
    alert_rate = len(alerts) / total_events if total_events else 0
```

```

metric_one, metric_two, metric_three, metric_four = st.columns(4)

metric_one.metric("Events analysed", f"{total_events:,}")
metric_two.metric("High / critical alerts", f"{high_risk_alerts:,}")
metric_three.metric("Alert rate", percent(alert_rate))
metric_four.metric(
    "Hold-out detection F1",
    f"{metrics.get('f1_score', 0):.4f}",
)

st.divider()

left_column, right_column = st.columns(2)

with left_column:
    st.subheader("Risk-score distribution")

    risk_chart = px.histogram(
        filtered_events,
        x="risk_score",
        color="risk_tier",
        nbins=30,
        category_orders={
            "risk_tier": ["Low", "Medium", "High", "Critical"]
        },
        color_discrete_map=RISK_COLORS,
        labels={
            "risk_score": "Risk score",
            "risk_tier": "Risk tier",
        },
    )
    risk_chart.add_vline(
        x=minimum_risk,
        line_dash="dash",
        line_color="red",
        annotation_text="Alert threshold",
    )
    risk_chart.update_layout(
        legend_title_text="Risk tier",
        xaxis_range=[0, 100],
    )
    st.plotly_chart(risk_chart, use_container_width=True)

with right_column:
    st.subheader("Predicted attack categories")

    if alerts.empty:
        st.info("No alerts match the current filters.")
    else:
        attack_counts = (
            alerts["predicted_attack_type"]
            .value_counts()
            .reset_index()
        )
        attack_counts.columns = ["Attack type", "Alerts"]

        attack_chart = px.bar(
            attack_counts,
            x="Alerts",
            y="Attack type",
            orientation="h",
            text="Alerts",
            color="Alerts",
            color_continuous_scale="Reds",
        )
        attack_chart.update_layout(
            yaxis={"categoryorder": "total ascending"},
            coloraxis_showscale=False,
        )
        st.plotly_chart(attack_chart, use_container_width=True)

st.subheader("Alert timeline")

if not alerts.empty:
    hourly_alerts = (
        alerts.set_index("timestamp")
        .resample("h")
        .size()
        .reset_index(name="alerts")
    )

    timeline_chart = px.area(
        hourly_alerts,
        x="timestamp",
        y="alerts",
        labels={
            "timestamp": "Time",
            "alerts": "Number of alerts",
        },
    )
    timeline_chart.update_traces(
        line_color="#E76F51",
        fillcolor="rgba(231, 111, 81, 0.25)",
    )
    st.plotly_chart(timeline_chart, use_container_width=True)
else:
    st.info("Adjust the filters to show an alert timeline.")

st.subheader("Ranked alert queue")

display_columns = [
    "event_id",
    "timestamp",
    "entity_id",
    "entity_type",
    "risk_score",
    "risk_tier",
    "predicted_attack_type",
    "geo_location",
    "resource_accessed",
    "explanation",
]

st.dataframe(
    alerts[display_columns].head(100),
    use_container_width=True,
    hide_index=True,
    column_config={
        "risk_score": st.column_config.ProgressColumn(
            "Risk score",
            min_value=0,
            max_value=100,

```

```

        format="%%.1f",
    ),
    "timestamp": st.column_config.DatetimeColumn(
        "Timestamp",
        format="YYYY-MM-DD HH:mm:ss",
    ),
    "explanation": st.column_config.TextColumn(
        "Contributing factors",
        width="large",
    ),
),
),
)

if alerts.empty:
    return

st.subheader("Investigate an alert")

alert_options = alerts.head(100).copy()
selected_event_id = st.selectbox(
    "Select an alert event",
    options=alert_options["event_id"].tolist(),
    format_func=lambda event_id: (
        f"{event_id} ⚠️" if
        f"{alert_options.loc[alert_options['event_id'] == event_id, 'entity_id'].iloc[0]} "
        f"(risk {alert_options.loc[alert_options['event_id'] == event_id, 'risk_score'].iloc[0]:.1f})"
    ),
)

selected_alert = alerts[
    alerts["event_id"] == selected_event_id
].iloc[0]

detail_left, detail_right = st.columns([1, 2])

with detail_left:
    st.metric("Risk score", f"{selected_alert['risk_score']:.1f}")
    st.metric("Risk tier", selected_alert["risk_tier"])
    st.metric(
        "Predicted attack",
        selected_alert["predicted_attack_type"].replace("_", " ").title(),
    )
    st.metric("Entity", selected_alert["entity_id"])

with detail_right:
    st.markdown("#### Why was this flagged?")
    st.info(selected_alert["explanation"])

    st.markdown("#### Event context")
    context_columns = [
        "timestamp",
        "source_ip",
        "geo_location",
        "resource_accessed",
        "auth_method",
        "auth_result",
        "device_fingerprint",
        "command_sequence",
        "bytes_transferred",
    ]
    st.dataframe(
        pd.DataFrame(selected_alert[context_columns]).T,
        use_container_width=True,
        hide_index=False,
    )

st.markdown("#### Entity behaviour history")

entity_history = events[
    events["entity_id"] == selected_alert["entity_id"]
].sort_values("timestamp")

history_chart = px.scatter(
    entity_history,
    x="timestamp",
    y="risk_score",
    color="risk_tier",
    hover_data=[
        "resource_accessed",
        "geo_location",
        "predicted_attack_type",
    ],
    category_orders={
        "risk_tier": ["Low", "Medium", "High", "Critical"]
    },
    color_discrete_map=RISK_COLORS,
    labels={
        "timestamp": "Time",
        "risk_score": "Risk score",
        "risk_tier": "Risk tier",
    },
)

history_chart.add_hline(
    y=minimum_risk,
    line_dash="dash",
    line_color="red",
    annotation_text="Alert threshold",
)

history_chart.update_yaxes(range=[0, 100])
st.plotly_chart(history_chart, use_container_width=True)

profile = profiles[
    profiles["entity_id"] == selected_alert["entity_id"]
]

if not profile.empty:
    st.markdown("#### Learned normal baseline")
    st.dataframe(profile, use_container_width=True, hide_index=True)

st.download_button(
    "Download filtered alert queue as CSV",
    data=alerts.to_csv(index=False).encode("utf-8"),
    file_name="sentinelai_alert_queue.csv",
    mime="text/csv",
)

if __name__ == "__main__":
    main()

```